

# Hisan Labs — Interview Preparation Guide

Prepared for: Sanket Ajay Chopade | DevOps Engineer

Based on: Hisan Labs website (hisanlabs.in) & Resume

---

## 1. About Hisan Labs

---

### Q1. What does Hisan Labs do, and what is their main area of focus?

#### Answer:

Hisan Labs is a cloud consulting and automation company based in India. Their main focus is helping startups and small-to-medium businesses migrate to AWS and set up cloud infrastructure. They specialise in:

- Cloud Migration — moving software to AWS using Docker and Kubernetes
- GitOps — managing infrastructure through Git for better control and security
- CI/CD and Release Engineering — automating software delivery pipelines
- Security and Compliance — making sure systems meet standards like SOC2, HIPAA, PCI/DSS
- Site Reliability Engineering (SRE) — monitoring systems and reducing downtime
- Automated Deployments — zero-downtime rollouts with one-click deployments

*Tip: Mention that their approach aligns with your work — you used all of these technologies during your internship.*

### Q2. Why did you want to intern at Hisan Labs specifically?

#### Answer:

Hisan Labs works on real-world AWS cloud projects and focuses on DevOps best practices like GitOps, CI/CD, and Infrastructure as Code. This was a perfect match for what I wanted to learn. I was looking for hands-on exposure to production systems, and Hisan Labs offered exactly that — working on a live e-commerce platform with a small, focused team.

*Tip: Keep this genuine and specific. Mention their cloud-first approach and small team structure.*

## 2. Your Work at Hisan Labs

---

### Q3. Can you describe the project you worked on at Hisan Labs?

#### Answer:

I worked on a microservices-based e-commerce application deployed on AWS. My responsibilities covered the full DevOps lifecycle:

- Provisioning AWS infrastructure (EKS, VPC, RDS, S3, EC2) using Terraform modules
- Building and maintaining 3 Jenkins CI/CD pipelines for Terraform, frontend, and backend deployments
- Containerising applications with Docker and deploying them on Amazon EKS using Kubernetes
- Integrating security tools like SonarQube, Trivy, and OWASP into the pipelines
- Setting up monitoring using Prometheus, Grafana, Datadog, and CloudWatch

*Tip: Walk through this confidently. You can say 'Our team of 4 handled the full deployment lifecycle.'*

### Q4. How did your CI/CD pipelines work at Hisan Labs?

#### Answer:

I designed and maintained 3 Jenkins pipelines:

- Pipeline 1 — Terraform infrastructure provisioning (VPCs, EKS clusters, RDS, S3)
- Pipeline 2 — Frontend deployment to AWS (static assets, CloudFront integration)
- Pipeline 3 — Backend microservices deployment to Kubernetes on EKS
- All pipelines were triggered via GitHub webhooks
- SonarQube was integrated to block non-compliant code before it reached staging or production

*Tip: Be ready to explain what a 'quality gate' means — it's a rule that stops a build if code quality checks fail.*

### **Q5. How did you manage Terraform infrastructure safely in a team?**

#### **Answer:**

We used S3-backed Terraform remote state with state locking. This means:

- The Terraform state file (which tracks infrastructure) was stored in an S3 bucket — not locally
- State locking prevented two team members from making changes at the same time
- We used Terraform workspaces to separate environments (dev, staging, production)
- All resources were written as reusable modules for consistency across environments

*Tip: If asked 'Why is remote state important?' — answer: 'To avoid conflicts when multiple people work on the same infrastructure.'*

### **Q6. What security practices did you follow in your CI/CD pipeline?**

#### **Answer:**

Security was built into every stage of the pipeline (DevSecOps approach):

- SonarQube — static code analysis to catch bugs and code quality issues early
- Trivy — scanned Docker container images for known vulnerabilities before deployment
- OWASP Dependency-Check — identified vulnerable third-party libraries in the codebase
- AWS IAM — applied least-privilege principles so each service/user had only the access it needed
- All security checks ran automatically before any code reached staging or production

*Tip: The key phrase interviewers love is 'shift-left security' — meaning catching issues early in the pipeline.*

## **3. Technical Questions Aligned with Hisan Labs' Services**

---

### **Q7. Hisan Labs uses GitOps heavily. Can you explain how GitOps works?**

#### **Answer:**

GitOps means using Git as the single source of truth for your infrastructure and application state. Here is how I used it:

- All Kubernetes manifests and Helm charts were stored in a Git repository
- Argo CD continuously watched the Git repository for changes
- When a developer pushed a change, Argo CD automatically synced the Kubernetes cluster to match
- If someone manually changed something in the cluster (drift), Argo CD detected it and reverted it
- This gave full audit trails — every change was tracked in Git history

*Tip: Hisan Labs lists GitOps as a core service. Show confidence here — you have real experience with Argo CD.*

### **Q8. Hisan Labs specialises in Cloud Migration. What AWS services have you worked with?**

#### **Answer:**

During my internship, I worked with a wide range of AWS services:

- EKS — managed Kubernetes service to run containerised workloads

- EC2 — virtual machines for compute needs
- VPC — network isolation with subnets, route tables, and security groups
- RDS — managed relational database service
- S3 — object storage for Terraform state, static assets, and logs
- ECR — private Docker image registry
- ALB — Application Load Balancer for routing traffic to services
- CloudWatch — AWS-native monitoring and log collection
- IAM — identity and access management for secure permissions

*Tip: Hisan Labs focuses on helping companies migrate to AWS. Showing breadth across these services is a strong signal.*

### **Q9. Hisan Labs mentions zero-downtime deployments. How did you achieve that?**

#### **Answer:**

In Kubernetes, zero-downtime deployments are achieved through Rolling Updates:

- Kubernetes replaces old pods with new ones gradually — never taking everything down at once
- Readiness probes ensure a new pod only receives traffic when it is fully ready
- If a deployment fails, Kubernetes automatically rolls back to the last stable version
- Horizontal Pod Autoscalers (HPA) scaled pods up or down based on CPU/memory usage
- The AWS Load Balancer (ALB) routes traffic only to healthy pods at all times

*Tip: Mention that you configured HPAs and monitored deployments in Grafana to verify zero-downtime behaviour.*

### **Q10. How did you monitor systems at Hisan Labs? (Relates to their SRE service)**

#### **Answer:**

We used a combination of monitoring tools to maintain full visibility:

- Prometheus — collected metrics from Kubernetes nodes, pods, and applications
- Grafana — visualised metrics in dashboards for real-time and historical views
- Datadog — provided deeper APM (Application Performance Monitoring) and alerting
- AWS CloudWatch — monitored AWS-specific resources like RDS, EC2, and EKS
- All tools were deployed using Helm charts for easy management and upgrades

*Tip: If asked about SLOs/SLAs (mentioned on Hisan Labs site), explain: 'SLO = target, e.g. 99.9% uptime. SLA = the contract with the client based on that target.'*

## **4. Behavioural / Situational Questions**

---

### **Q11. Tell me about a challenge you faced at Hisan Labs and how you solved it.**

#### **Answer:**

One challenge was preventing Terraform state conflicts when multiple team members ran deployments simultaneously. The solution I implemented:

- Moved the state file from local storage to an S3 bucket
- Enabled DynamoDB-based state locking to prevent concurrent modifications
- Created separate Terraform workspaces for dev, staging, and production
- Documented the workflow so all 4 team members followed the same process

*Tip: Use the STAR method — Situation, Task, Action, Result. End with the positive outcome: 'This eliminated state conflicts entirely for the rest of the internship.'*

### **Q12. Hisan Labs works with small teams on big cloud projects. How did you collaborate effectively?**

### Answer:

As part of a 4-member DevOps team, I focused on clear communication and documentation:

- Used Git branching strategies so everyone worked on separate branches and reviewed each other's code
- Wrote reusable Terraform modules so any team member could use them without re-learning the setup
- Used Jenkins pipeline-as-code (Jenkinsfiles) stored in Git — visible and editable by all
- Documented operational scripts and runbooks for recurring tasks to reduce toil

*Tip: Hisan Labs values 'collaboration without sacrificing security' — mention that Git was the central source of truth for everything.*

### Q13. How does your internship experience align with what Hisan Labs does as a company?

#### Answer:

My internship gave me direct experience in everything Hisan Labs offers as a service:

- Cloud Migration — I provisioned and managed full AWS environments with Terraform
- GitOps — I implemented Argo CD for automated, Git-driven Kubernetes deployments
- CI/CD and Release Engineering — I built 3 Jenkins pipelines that automated the full release cycle
- Security and Compliance — I integrated DevSecOps tools (SonarQube, Trivy, OWASP) into pipelines
- Site Reliability — I set up monitoring with Prometheus, Grafana, Datadog, and CloudWatch

*Tip: This question is your strongest opportunity. Show that you did not just use these tools — you understand why they matter to Hisan Labs' clients.*

---

## General Interview Tips

- Research Hisan Labs' webinars and articles on their website before the interview — showing awareness of their content stands out.
- Use specific numbers where possible: '3 pipelines', '4-member team', '9 months experience'.
- Be ready to whiteboard or explain a Jenkins pipeline or Kubernetes deployment flow step by step.
- Hisan Labs values security — always link your answers back to secure practices (least privilege, DevSecOps, GitOps audit trails).
- If asked about a tool you haven't used deeply, be honest — then say what you would do to learn it quickly.